



MODUL 12 · AUTHENTICATION & AUTHORIZATION

Role-Based Access Control



Authorization berdasarkan role. Siapa boleh melakukan apa di halaman dan API.

Setup Role di User Model & Better Auth

Tambahkan field role ke User. Better Auth plugin untuk RBAC, atau custom role check.

Schema: tambah role ke User

```
// prisma/schema.prisma
model User {
  id          String    @id
  name        String
  email       String    @unique
  emailVerified Boolean
  image       String?
  role        UserRole  @default(READER) // ← tambahkan
  createdAt   DateTime
  updatedAt   DateTime
  sessions    Session[]
  accounts    Account[]
  @@map("user")
}

enum UserRole {
  READER // bisa baca content
  AUTHOR // bisa buat + edit post sendiri
  ADMIN  // akses penuh
}

// Setelah update schema:
// bunx prisma migrate dev --name add_role

// Set role via Prisma:
// await db.user.update({
//   where: { email: "admin@app.com" },
//   data: { role: "ADMIN" }
// })
```

lib/auth.ts – custom role field

```
import { betterAuth } from "better-auth"
import { prismaAdapter } from "better-auth/adapters/prisma"
import { nextCookies } from "better-auth/next-js"

export const auth = betterAuth({
  database: prismaAdapter(db, { provider: "postgresql" }),
  plugins: [nextCookies()],

  emailAndPassword: { enabled: true },

  // Better Auth: custom user fields
  user: {
    additionalFields: {
      role: {
        type: "string",
        default: "READER",
        // Tambahkan ke session agar bisa diakses
        // tanpa query DB tambahan
        required: false,
      },
    },
  },
})

// Type update agar TypeScript tahu role ada:
export type Session = typeof auth.$Infer.Session
```

Role Check di Halaman, Action & API

Authorization harus dicek di setiap layer — bukan hanya di UI.

● ● ● Role check di setiap layer

```
// — Server Component (halaman admin) —————
export default async function AdminPage() {
  const session = await auth.api.getSession({ headers: await headers() })
  if (!session) redirect("/login")
  if (session.user.role !== "ADMIN") redirect("/unauthorized")
  return <AdminDashboard />
}

// — Server Action (admin mutation) —————
export async function deleteUser(userId: string) {
  "use server"
  const session = await auth.api.getSession({ headers: await headers() })
  if (!session) return { error: "Unauthorized" }
  if (session.user.role !== "ADMIN") return { error: "Forbidden — Admin only" }
  await db.user.delete({ where: { id: userId } })
  return { success: true }
}

// — Author check — bisa aksi sendiri atau admin —————
export async function updatePost(postId: string, data: UpdatePostInput) {
  "use server"
  const session = await auth.api.getSession({ headers: await headers() })
  if (!session) return { error: "Unauthorized" }

  const isAdmin = session.user.role === "ADMIN"
  const isAuthor = await db.post.findFirst({
    where: { id: postId, authorId: session.user.id }
  })

  if (!isAdmin && !isAuthor) return { error: "Forbidden" }
  return { data: await db.post.update({ where: { id: postId }, data }) }
}
```

Conditional UI Berdasarkan Role

Tampilkan/sembunyikan UI berdasarkan role. Tapi UI hiding BUKAN security — cek tetap di server.

Conditional UI — Server Component

```
// — Server Component: role dari session
export default async function PostDetailPage({ params }) {
  const { id } = await params
  const session = await auth.api.getSession({ headers: await headers() })
  const post = await db.post.findUnique({ where: { id } })
  if (!post) notFound()

  const isAdmin = session?.user.role === "ADMIN"
  const isAuthor = session?.user.id === post.authorId

  return (
    <article>
      <h1>{post.title}</h1>
      <p>{post.content}</p>

      {/* Tombol edit hanya untuk author + admin */}
      {(isAuthor || isAdmin) && (
        <div className="flex gap-2 mt-4">
          <a href={`\dashboard/posts/${id}/edit`} >
            <Button variant="outline">Edit</Button>
          </a>
          {isAdmin && (
            <DeleteButton postId={id} /> // ← hanya admin bisa hapus
          )}
        </div>
      )}
    </article>
  )
}

// ⚠️ INGAT: Menyembunyikan tombol di UI BUKAN security!
// User bisa tetap call Server Action langsung.
// Security yang nyata ada di server action deletePost()
// yang cek session.user.role === "ADMIN"
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Mengapa menyembunyikan tombol 'Hapus' di UI berdasarkan role BUKAN merupakan security yang cukup?

- A) Karena CSS tidak bisa di-trust B) User bisa tetap call Server Action deletePost() langsung tanpa melalui UI C) Karena tombol tetap visible di source code

Q2 — Urutan check yang benar di Server Action untuk RBAC?

- A) Ownership → Role → Auth B) Auth (session) → Role → Ownership → Execute C) Role → Execute (role sudah cukup)

Q3 — Kenapa role sebaiknya dimasukkan ke session (additionalFields) di Better Auth?

- A) Lebih aman dari role di database B) Menghindari query DB tambahan setiap request — role tersedia langsung dari session C) Required oleh Better Auth

Tugas Mandiri

Implementasikan RBAC lengkap untuk dashboard blog.

01 Schema + Migrate

Tambahkan enum UserRole dan field role ke User model. Migrate. Set role ADMIN untuk satu user via Prisma Studio atau seed.

02 Admin Pages

Buat app/admin/page.tsx yang hanya ADMIN bisa akses. Cek session.user.role !== 'ADMIN' → redirect ke /unauthorized.

03 Author Actions

Update updatePost dan deletePost Server Actions: ADMIN bisa semua, AUTHOR hanya miliknya, READER tidak bisa mutasi.

04 Conditional UI

Di PostCard: tampilkan tombol Edit hanya jika isAuthor || isAdmin. Tampilkan tombol Delete hanya jika isAdmin. Ingat: ini UX, bukan security!

💡 Test RBAC: login sebagai READER → coba akses /admin → redirect. Login sebagai ADMIN → semua aksi tersedia.

Yang Sudah Kita Pelajari

- ✓ RBAC: tambahkan enum UserRole + field role ke User. READER, AUTHOR, ADMIN.
- ✓ additionalFields di Better Auth: include role di session — no extra DB query.
- ✓ Auth → Role check → Ownership check → Execute. Urutan ini wajib di setiap Server Action.
- ✓ Conditional UI berdasarkan role = UX yang baik. BUKAN security — tetap cek di server!
- ✓ ADMIN bypass ownership. AUTHOR hanya ke miliknya. READER read-only.

→ Selanjutnya: **Modul 13 — Upload File & Optimasi Gambar**

